



Guest Check-In System

User Manual & Technical Reference

Version 1.0.0

Date June 2025

Status Production Release

Table of Contents

- 3. Executive Summary
- 4. High-Level Architecture
- 5. Process Flow — State Machine
- 6. Technical Stack
- 7. Non-Technical Stack
- 8. System Requirements
- 9. Deployment Guide
- 10. Configuration Reference
- 11. API Reference
- 12. Widget Integration Guide
- 13. State Machine Reference
- 14. MCP Tools Reference
- 15. Troubleshooting
- 16. Glossary
- 17. Appendix: Database Schema

3. Executive Summary

The **Guest Check-In System** is a conversation-based, AI-powered onboarding platform for short-term rental properties. It replaces fragmented email-link workflows with a single, guided chat experience that walks every guest through the required check-in steps — privacy policy acceptance, house rules, rental agreement, identity verification, ID upload, and incidental protection — in a natural conversation.

Who It's For

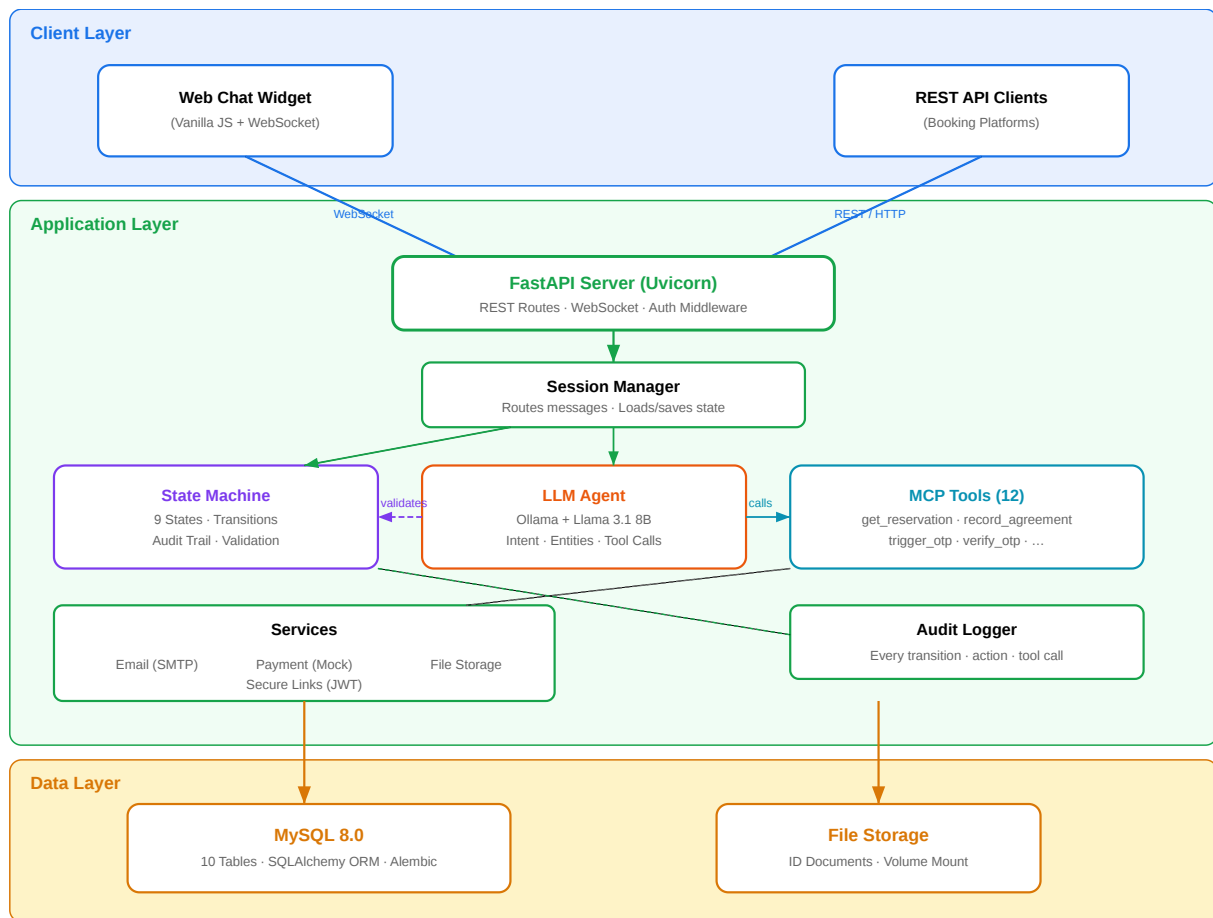
- **Property Managers & Booking Platforms** — Integrate via REST API or embed the chat widget on any website.
- **Guests** — Complete onboarding by chatting with an AI agent; no app download or portal login required.

Key Benefits

- **Higher Completion Rates** — Guided conversation prevents guest abandonment at any step.
- **Real-Time Assistance** — Guests can ask questions mid-process; the AI answers from the property FAQ.
- **Full Audit Trail** — Every agreement, transition, and action is recorded with timestamps.
- **Channel-Agnostic** — REST API + WebSocket + embeddable widget work with any booking channel.
- **Local LLM** — Runs Ollama (Llama 3.1 8B) on-premise; no cloud API keys, no per-message cost.
- **State-Machine Safety** — The AI agent can never skip a step; the state machine is the single source of truth.

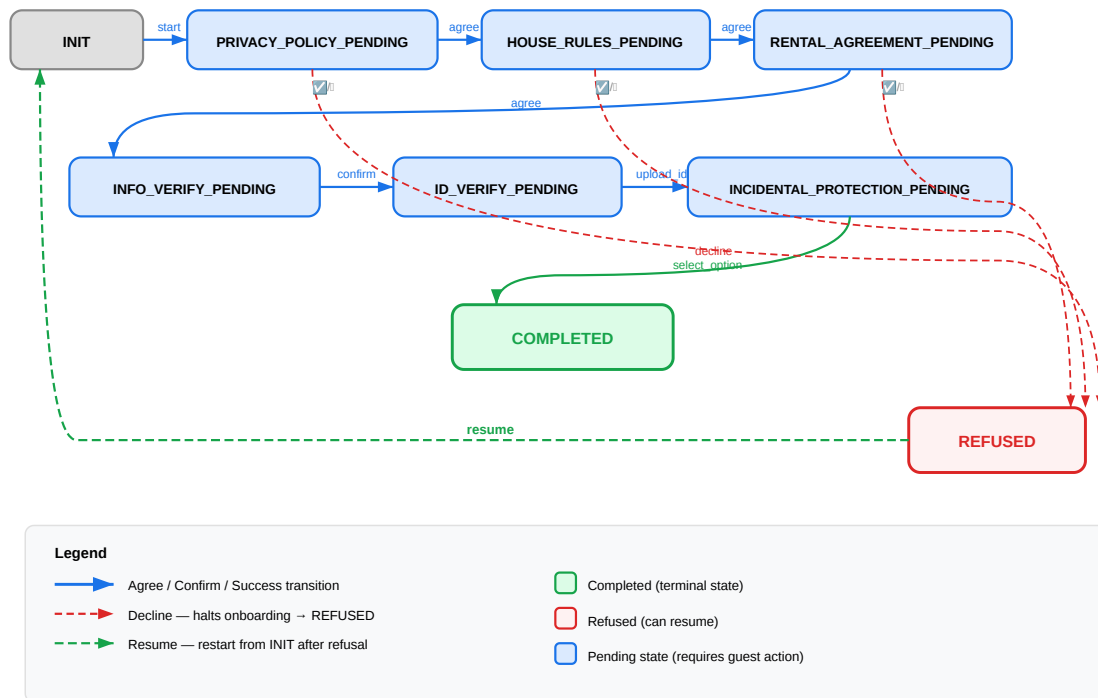
4. High-Level Architecture

The system follows a three-layer architecture: **Client Layer** (web widget + API clients), **Application Layer** (FastAPI server, state machine, LLM agent, MCP tools, services), and **Data Layer** (MySQL + file storage).



5. Process Flow — State Machine

The guest onboarding workflow is governed by a deterministic state machine with **9 states**. The LLM agent detects intent and calls MCP tools, but the state machine validates every transition — the agent can never skip a step.



At each agreement step (Privacy Policy, House Rules, Rental Agreement), the guest can **agree** (advancing to the next state) or **decline** (transitioning to REFUSED). A refused session can be **resumed** at any time, restarting from the beginning (INIT).

6. Technical Stack

Component	Technology	Version	License
Web Framework	FastAPI	0.110+	MIT
ASGI Server	Uvicorn	0.27+	BSD
ORM	SQLAlchemy	2.0+	MIT
Migrations	Alembic	1.13+	MIT
MySQL Driver (sync)	PyMySQL	1.1+	MIT
MySQL Driver (async)	aiomysql	0.2+	MIT
Database	MySQL	8.0	GPL
LLM Runtime	Ollama	Latest	MIT
LLM Model	Llama 3.1 8B Instruct	8B	Llama 3.1 License
Email (Dev)	MailHog	v1.0.1	MIT
Template Engine	Jinja2	3.1+	BSD
Validation	Pydantic	v2 (2.5+)	MIT
Settings	pydantic-settings	2.1+	MIT
Rate Limiting	slowapi	0.1.9+	MIT

Component	Technology	Version	License
HTTP Client	httpx	0.26+	BSD
Async SMTP	aiosmtplib	3.0+	MIT
Testing	pytest	8.0+	MIT
Async Testing	pytest-asyncio	0.23+	MIT
E2E Testing	Playwright (Python)	Latest	Apache 2.0
Containerization	Docker + Docker Compose	3.9	Apache 2.0

7. Non-Technical Stack

What each component *does* in plain language:

Capability	Tool	Purpose
LLM (Ollama)	Llama 3.1 8B	Understands guest messages, detects intent (agree, decline, question), extracts entities, and decides which tool to call.
State Machine	Custom Python	Controls the onboarding workflow sequence — ensures guests complete steps in order; the LLM cannot skip or reorder.
Database (MySQL)	MySQL 8.0	Stores all guest, reservation, session, agreement, OTP, payment, message, and audit data persistently.
Email (SMTP)	MailHog (dev) / SMTP	Sends OTP verification codes to guests' email addresses for identity confirmation.
Web Server	FastAPI + Uvicorn	Handles HTTP requests and WebSocket connections from clients and booking platforms.
Authentication	API Key + Session Token	API keys secure platform endpoints; session tokens secure guest-facing endpoints (ID upload, OTP, payment).
File Storage	Local filesystem	Saves uploaded ID documents; volume-mounted in Docker for persistence.
Payment	MockPayment (v1)	Simulates payment processing for incidental protection; swappable with real gateway (Stripe, etc.).
Audit Trail	Custom logger + DB	Records every state transition, tool call, and guest action with timestamps — immutable log.
Containerization	Docker Compose	Packages app, MySQL, and MailHog into reproducible, one-command deployment.

8. System Requirements

Hardware

Resource	Minimum	Recommended
CPU	2 cores	4+ cores (for Ollama inference)
RAM	4 GB	8 GB+ (Ollama + Llama 3.1 8B requires ~6 GB)
Disk	20 GB	50 GB+ (Ollama model ~4.7 GB + Docker images + data)
GPU	Not required (CPU inference)	NVIDIA GPU with 8 GB+ VRAM for faster responses

Software

- **Docker** 20.10+ and **Docker Compose** v2+
- **Ollama** (installed on host or separate machine)
- Operating system: Linux (Ubuntu 20.04+), macOS, or Windows with WSL2

Network

- Port **8000** — Application (FastAPI / Uvicorn)
- Port **3306** — MySQL
- Port **1025** — MailHog SMTP
- Port **8025** — MailHog Web UI
- Port **11434** — Ollama API

9. Deployment Guide

9.1 Prerequisites

1. **Docker** — `docker --version` should return 20.10+
2. **Docker Compose** — `docker compose version` should return v2+
3. **Ollama** — Install from ollama.com and verify: `ollama --version`

9.2 Clone the Repository

```
git clone https://github.com/your-org/guest-checkin.git
cd guest-checkin
```

9.3 Configure Environment

```
cp .env.example .env
# Edit .env with your settings (see Section 10 for full reference)
```

At minimum, set `DATABASE_URL`, `OLLAMA_BASE_URL`, and `LINK_SECRET` (change from the default for production).

9.4 Pull the Ollama Model

```
ollama pull llama3.1:8b
```

Note: This downloads ~4.7 GB. Run `ollama serve` first if the daemon is not already running.

9.5 Start Services

```
docker compose up -d --build
```

This starts three containers:

- **app** — FastAPI on port 8000
- **mysql** — MySQL 8.0 on port 3306

- **mailhog** — SMTP on port 1025, Web UI on port 8025

9.6 Run Database Migrations

```
docker compose exec app alembic upgrade head
```

9.7 Seed the Database

```
docker compose exec app python -m seed.seed_reservations
docker compose exec app python -m seed.seed_knowledge_base
docker compose exec app python -m seed.seed_api_keys
```

9.8 Verify the System

```
# Health check
curl http://localhost:8000/docs

# Test with demo API key
curl -H "X-API-Key: demo-key" \
  http://localhost:8000/api/v1/reservations/BK-2024-001

# Check MailHog
open http://localhost:8025
```

If all return successful responses, the system is running.

10. Configuration Reference

All settings are loaded from environment variables or a `.env` file via `pydantic-settings`.

Variable	Description	Default
<code>DATABASE_URL</code>	SQLAlchemy connection string for MySQL	<code>mysql+pymysql://guestcheckin:guestcheckin@localhost:3306/guestcheckin</code>
<code>OLLAMA_BASE_URL</code>	Base URL of the Ollama API	<code>http://localhost:11434</code>
<code>OLLAMA_MODEL</code>	Model tag to use for inference	<code>llama3.1:8b</code>
<code>SMTP_HOST</code>	SMTP server hostname	<code>mailhog (Docker) / localhost (local)</code>
<code>SMTP_PORT</code>	SMTP server port	<code>1025</code>
<code>SMTP_USER</code>	SMTP username (empty for MailHog)	<code>empty</code>
<code>SMTP_PASS</code>	SMTP password (empty for MailHog)	<code>empty</code>
<code>SMTP_FROM</code>	Sender email address for	<code>checkin@guestapp.local</code>

Variable	Description	Default
	outgoing mail	
UPLOAD_DIR	Filesystem path for ID document uploads	/app/uploads
LINK_SECRET	HMAC secret for secure link tokens — change in production!	change-me-in-production
LINK_EXPIRY_HOURS	Hours before secure upload/payment links expire	1
API_KEY_HEADER	HTTP header name for API key authentication	X-API-Key
RATE_LIMIT	Rate limit per API key (slowapi format)	60/minute
CORS_ORIGINS	JSON array of allowed CORS origins for the chat widget	["http://localhost:3000", "http://localhost:8000"]

Security: Always change `LINK_SECRET` in production. Use a strong random string (32+ characters).

11. API Reference

Base URL: `http://localhost:8000/api/v1`

Authentication: Endpoints marked **API Key** require the `x-API-Key` header. Endpoints marked **Token** require `Authorization: Bearer <session_token>`.

POST /sessions [API Key](#)

Create a new check-in session for a booking reference.

Request Body

```
{ "booking_reference": "BK-2024-001" }
```

Response 201

```
{
  "id": "a1b2c3d4-...",
  "reservation_id": "r5e6f7-...",
  "guest_id": "g8h9i0-...",
  "current_state": "INIT",
  "session_token": "tok_abc123...",
  "status": "active",
  "created_at": "2025-06-10T12:00:00Z",
  "updated_at": "2025-06-10T12:00:00Z"
}
```

Error Responses

- 404 — Reservation not found for the given booking reference.
- 401 — Invalid or missing API key.

GET /sessions/{session_id} [API Key](#)

Retrieve session status, current state, and metadata.

Response 200

```
{
  "id": "a1b2c3d4-...",
  "reservation_id": "r5e6f7-...",
  "guest_id": "g8h9i0-...",
  "current_state": "PRIVACY_POLICY_PENDING",
  "session_token": "tok_abc123...",
  "status": "active",
  "created_at": "2025-06-10T12:00:00Z",
  "updated_at": "2025-06-10T12:01:00Z"
}
```

- 404 — Session not found.

POST /sessions/{session_id}/messages [API Key](#)

Send a guest message and receive the agent's response. The SessionManager processes the message through intent detection → MCP tool calls → state machine transition.

Request Body

```
{ "content": "I agree to the privacy policy" }
```

Response 200

```
{
  "message": {
    "id": "m1n2o3-...",
    "session_id": "a1b2c3d4-...",
    "role": "agent",
    "content": "Thank you! Now let's review the house rules...",
    "intent_detected": "agree",
    "tools_called": ["record_agreement"],
    "created_at": "2025-06-10T12:02:00Z"
  },
  "current_state": "HOUSE_RULES_PENDING",
  "required_action": "Accept or decline the House Rules",
  "session_status": "active"
}
```

404 — Session not found.

422 — Invalid state transition.

503 — Service temporarily unavailable (LLM or DB error).

GET /sessions/{session_id}/messages [API Key](#)

Retrieve the full message history for a session, ordered chronologically.

Response 200

```
[
  { "id": "msg1", "session_id": "...", "role": "agent",
    "content": "Welcome! ...", "intent_detected": null,
    "tools_called": null, "created_at": "..." },
  { "id": "msg2", "session_id": "...", "role": "guest",
    "content": "I agree", "intent_detected": "agree",
    "tools_called": ["record_agreement"], "created_at": "..." }
]
```

GET /sessions/{session_id}/state [API Key](#)

Get the current state machine state and required action.

Response 200

```
{
  "session_id": "a1b2c3d4-...",
  "current_state": "PRIVACY_POLICY_PENDING",
  "required_action": "Accept or decline the Privacy Policy & Data Usage agreement",
  "status": "active"
}
```

POST /sessions/{session_id}/resume [API Key](#)

Restart a REFUSED session from the beginning (INIT state).

Response 200

```
{
  "session_id": "a1b2c3d4-...",
  "current_state": "INIT",
  "required_action": "Initialize check-in session",
  "status": "active"
}
```

422 — Session is not in REFUSED state (cannot resume).

GET /reservations/{booking_reference} [API Key](#)

Look up a reservation by booking reference.

Response 200

```
{
  "id": "r5e6f7-...",
  "booking_reference": "BK-2024-001",
  "property_name": "Seaside Cottage",
  "property_address": "123 Ocean Drive, Malibu, CA",
  "guest_name": "Jane Smith",
  "guest_email": "jane@example.com",
  "guest_phone": "+1-555-0123",
  "check_in_date": "2025-07-01",
  "check_out_date": "2025-07-05",
  "num_guests": 2,
  "wifi_network": "SeasideGuest",
  "wifi_password": "ocean123",
  "lockbox_code": "4521",
  "emergency_contact": "+1-555-9999",
  "created_at": "...", "updated_at": "..."
}
```

404 — Reservation not found.

POST /otp/trigger [Token](#)

Generate and send a 6-digit OTP to the guest's email. Requires session token auth.

Response 200

```
{
  "message": "OTP sent",
  "email": "j***@example.com",
  "expires_at": "2025-06-10T12:10:00Z"
}
```

POST /otp/verify **Token**

Verify a 6-digit OTP code. Maximum 3 attempts per code.

Request Body

```
{ "otp_code": "123456" }
```

Response 200

```
{
  "verified": true,
  "attempts_remaining": 2,
  "message": "OTP verified successfully."
}
```

404 — No active OTP found (expired or none triggered).

POST /id-upload/{token}

Upload an ID document via a secure, time-limited link token (generated by the agent). Accepts `multipart/form-data`.

Request

```
Content-Type: multipart/form-data
file: <document image/PDF>
```

Response 200

```
{
  "message": "ID document uploaded successfully",
  "file_path": "/app/uploads/ids/session_abc_passport.jpg",
  "verified": true
}
```

401 — Invalid or expired link token.

413 — File too large (>10 MB).

422 — Invalid file extension.

POST /incidental/{token}

Select incidental protection and process mock payment via secure link token.

Request Body

```
{ "selection_type": "damage_waiver" }
```

Response 200

```
{
  "selection_type": "damage_waiver",
  "amount": 49.99,
  "payment_status": "completed",
  "payment_reference": "txn_mock_abc123",
  "message": "Payment processed successfully."
}
```

401 — Invalid or expired link token.

422 — Invalid selection type.

GET /guests/{guest_id}/audit-trail [API Key](#)

Retrieve the full audit trail for a guest — all state transitions, tool calls, and actions.

Response 200

```
[
  {
    "id": "at1-...",
    "session_id": "...",
    "action": "state_transition",
    "from_state": "INIT",
    "to_state": "PRIVACY_POLICY_PENDING",
    "details": { "intent": "start" },
    "actor": "guest",
    "created_at": "2025-06-10T12:00:05Z"
  }
]
```

WS /ws/{session_id}?token={session_token}

Real-time bidirectional chat via WebSocket. Authenticate with the session token as a query parameter.

Connect

```
ws://localhost:8000/api/v1/ws/a1b2c3d4-...?token=tok_abc123...
```

Welcome Message (server → client on connect)

```
{ "type": "welcome", "data": {
  "session_id": "...",
  "current_state": "INIT",
  "required_action": "Initialize check-in session",
  "session_status": "active"
}}
```

Guest Sends

```
{ "content": "I agree to the privacy policy" }
```

Agent Responds

```
{ "type": "agent", "data": {
  "message": { "session_id": "...", "role": "agent",
    "content": "Thank you! ...", "intent_detected": "agree",
    "tools_called": ["record_agreement"] },
  "current_state": "HOUSE_RULES_PENDING",
  "required_action": "Accept or decline the House Rules",
  "session_status": "active"
}}
```

Error Message

```
{ "type": "error", "data": { "detail": "Invalid message format" } }
```

12. Widget Integration Guide

The guest check-in widget is a lightweight, embeddable chat bubble built with vanilla JavaScript. No React, Vue, or build step required.

12.1 Basic Integration

Add the CSS and JS to your page, then initialize the widget:

```
<!-- In the <head> of your page -->
<link rel="stylesheet" href="https://your-domain.com/static/chat-widget.css">

<!-- Before </body> -->
<script src="https://your-domain.com/static/chat-widget.js"></script>
<script>
  var widget = GuestCheckInWidget.init({
```



```
sessionId: 'SESSION_ID_FROM_API',
token: 'SESSION_TOKEN_FROM_API',
wsUrl: 'wss://your-domain.com/api/v1/ws/SESSION_ID?token=TOKEN',
apiUrl: 'https://your-domain.com/api/v1'
});
</script>
```

12.2 Configuration Options

Option	Type	Required	Description
sessionId	string	Yes	Session ID from <code>POST /sessions</code>
token	string	Yes	Session token for WebSocket auth
wsUrl	string	Yes	Full WebSocket URL including token query param
apiUrl	string	Yes	Base REST API URL (e.g. <code>https://your-domain.com/api/v1</code>)
theme	string	No	Widget theme: <code>"light"</code> (default) or <code>"dark"</code>

12.3 Event Callbacks

Subscribe to message events after initialization:

```
widget.callbacks.message.push(function(msg) {
  console.log('New message:', msg);
});
```

12.4 Customization

- **Theme** — Pass `theme: "dark"` to `init()` for dark mode.
- **Position** — The widget appears as a floating action button (FAB) in the bottom-right corner. Override CSS class `.gci-fab` and `.gci-widget` to reposition.
- **Progress Steps** — The widget shows a 6-step progress bar automatically mapping states to steps (Privacy → Rules → Rental → Info → ID → Incidental).
- **Auto-reconnect** — The WebSocket client reconnects automatically with exponential backoff (1s → 30s max) if the connection drops.

12.5 Typical Integration Flow

1. Your backend calls `POST /api/v1/sessions` with the booking reference and API key.
2. The response includes `id` (session ID) and `session_token`.
3. Pass these values to `GuestCheckInWidget.init()` on your frontend page.
4. The widget opens, connects via WebSocket, and the conversation begins.

13. State Machine Reference

Nine states govern the guest onboarding flow. The LLM agent detects intent and calls MCP tools; the state machine validates every transition.

State	Required Action	Valid Intents	Next (Success)	Next (Decline)
INIT	Initialize check-in session	start	PRIVACY_POLICY_PENDING	—
PRIVACY_POLICY_PENDING	Accept or decline the Privacy Policy & Data Usage	agree , decline	HOUSE_RULES_PENDING	REFUSED
HOUSE_RULES_PENDING	Accept or decline the House Rules	agree , decline	RENTAL_AGREEMENT_PENDING	REFUSED
RENTAL_AGREEMENT_PENDING	Accept or decline the Rental Agreement	agree , decline	INFO_VERIFY_PENDING	REFUSED
INFO_VERIFY_PENDING	Confirm personal information (name, email, phone, guests)	confirm , provide_info	ID_VERIFY_PENDING	— (stays in state on provide_info)
ID_VERIFY_PENDING	Upload government-issued ID via secure link	upload_id	INCIDENTAL_PROTECTION_PENDING	—
INCIDENTAL_PROTECTION_PENDING	Select Damage Waiver or Security Hold + payment	select_option	COMPLETED	—
COMPLETED	None — onboarding finished	— (terminal)	—	—
REFUSED	Guest declined an agreement; onboarding halted	resume	INIT (restart)	—

Key rule: The LLM agent *never* advances state directly. It detects intent and calls MCP tools. The state machine validates the tool result and transitions. This prevents hallucination from skipping steps.

14. MCP Tools Reference

The LLM agent has access to 12 MCP (Model Context Protocol) tools — internal Python functions that handle data operations. The agent calls these tools; the state machine validates results and transitions.

1. get_reservation

Description	Retrieve reservation details by booking reference. Returns property name, guest info, dates, Wi-Fi, lockbox code, and agreement texts.
Parameters	<code>booking_reference</code> (string, required) — e.g. "BK-2024-001"
Returns	Reservation object with all fields, or <code>null</code> if not found
Active In	Any state

2. record_agreement

Description	Record guest acceptance or refusal of an agreement (privacy policy, house rules, or rental agreement).
Parameters	<code>session_id</code> (string, required), <code>agreement_type</code> (string, required: "privacy_policy" "house_rules" "rental_agreement"), <code>accepted</code> (boolean, required), <code>guest_response</code> (string, optional)
Returns	<code>{ "recorded": true, "agreement_id": "...", "agreement_type": "...", "accepted": true/false }</code>
Active In	PRIVACY_POLICY_PENDING, HOUSE_RULES_PENDING, RENTAL_AGREEMENT_PENDING

3. update_guest_info

Description	Update guest personal information during the info verification step.
Parameters	<code>session_id</code> (string, required), <code>first_name</code> (string, optional), <code>last_name</code> (string, optional), <code>phone</code> (string, optional)
Returns	<code>{ "updated": true, "guest_id": "...", "updated_fields": [...], "first_name": "...", "last_name": "...", "email": "...", "phone": "..." }</code>
Active In	INFO_VERIFY_PENDING

4. trigger_otp

Description	Generate a 6-digit OTP and email it to the guest. Code expires in 10 minutes; max 3 verification attempts.
Parameters	<code>session_id</code> (string, required)
Returns	<code>{ "otp_sent": true, "email": "j***@example.com", "expires_in": "10 minutes" }</code>
Active In	INFO_VERIFY_PENDING

5. verify_otp

Description	Verify a 6-digit OTP code entered by the guest. Checks expiry and attempt count.
Parameters	<code>session_id</code> (string, required), <code>otp_code</code> (string, required)

Returns	<code>{ "verified": true/false, "attempts_remaining": N }</code>
Active In	INFO_VERIFY_PENDING

6. generate_id_upload_link

Description	Generate a secure, time-limited link (1 hour) for the guest to upload their government-issued ID.
Parameters	<code>session_id</code> (string, required)
Returns	<code>{ "upload_url": "/api/v1/id-upload?token=...", "expires_in": "1 hour(s)" }</code>
Active In	ID_VERIFY_PENDING

7. record_id_upload

Description	Record that the guest has uploaded their ID document. Updates guest record with file path and marks ID as verified.
Parameters	<code>session_id</code> (string, required), <code>file_path</code> (string, required)
Returns	<code>{ "uploaded": true, "guest_id": "...", "id_document_path": "...", "id_verified": true }</code>
Active In	ID_VERIFY_PENDING

8. generate_incidental_link

Description	Generate a secure, time-limited link for the guest to select incidental protection (Damage Waiver or Security Hold) and complete payment.
Parameters	<code>session_id</code> (string, required)
Returns	<code>{ "selection_url": "/api/v1/incidental/select?token=...", "expires_in": "1 hour(s)", "options": [...] }</code>
Active In	INCIDENTAL_PROTECTION_PENDING

9. record_incidental_selection

Description	Record the guest's incidental protection choice and process payment via the payment gateway.
Parameters	<code>session_id</code> (string, required), <code>selection_type</code> (string, required: "damage_waiver" "security_hold"), <code>amount</code> (number, optional)
Returns	<code>{ "selected": true, "selection_id": "...", "selection_type": "...", "amount": 49.00, "payment": { "success": true, "transaction_id": "..." } }</code>
Active In	INCIDENTAL_PROTECTION_PENDING

10. get_arrival_instructions

Description	Generate formatted arrival instructions (property address, Wi-Fi, lockbox code, emergency contact) from the Jinja2 template.
Parameters	<code>session_id</code> (string, required)
Returns	<code>{ "instructions_html": "...", "property_name": "...", "check_in_date": "...", ... }</code>

Active In	COMPLETED
-----------	-----------

11. get_faq_answer

Description	Search the property knowledge base for an answer to the guest's question using keyword matching.
Parameters	session_id (string, required), question (string, required)
Returns	<code>{ "answer": "...", "category": "amenities" }</code> or <code>{ "answer": "I don't have information about that.", "category": null }</code>
Active In	Any state

12. get_current_state

Description	Get the guest's current onboarding state and required action. Used by the agent to understand where the guest is in the flow.
Parameters	session_id (string, required)
Returns	<code>{ "state": "PRIVACY_POLICY_PENDING", "required_action": "...", "description": "..." }</code>
Active In	Any state

15. Troubleshooting

15.1 Ollama Not Running / Model Not Found

Symptoms: Agent responses are slow or use fallback (regex-based) responses. Logs show `Connection refused to localhost:11434`.

Solution:

1. Start Ollama: `ollama serve`
2. Pull the model: `ollama pull llama3.1:8b`
3. Verify: `curl http://localhost:11434/api/tags`
4. If running in Docker, ensure `OLLAMA_BASE_URL=http://host.docker.internal:11434`

Fallback: The system automatically falls back to regex-based intent matching when Ollama is unreachable. Guests can still complete onboarding, but responses will be less conversational.

15.2 MySQL Connection Refused

Symptoms: `503 Service Unavailable` on all API calls. Logs show `Can't connect to MySQL server`.

Solution:

1. Check container: `docker compose ps mysql`
2. Wait for healthcheck: MySQL can take 10-30 seconds on first start.
3. Verify credentials in `.env` match `docker-compose.yml`.
4. Test connection: `docker compose exec mysql mysql -uguestcheckin -pguestcheckin guestcheckin`

15.3 MailHog Not Receiving Emails

Symptoms: OTP emails are not appearing in the MailHog Web UI at `http://localhost:8025`.

Solution:

1. Check MailHog container: `docker compose ps mailhog`
2. Verify `SMTP_HOST=mailhog` and `SMTP_PORT=1025` in the app environment.
3. Test SMTP: `docker compose exec app python -c "import smtplib; s=smtplib.SMTP('mailhog',1025); s.sendmail('test@test.com','guest@example.com','Test'); print('OK')"`

15.4 WebSocket Disconnects

Symptoms: Chat widget shows "Disconnected" or stops receiving messages.

Solution:

1. The widget auto-reconnects with exponential backoff (1s → 30s). If it doesn't, refresh the page.
2. Check server logs for WebSocket errors: `docker compose logs app | grep WebSocket`
3. Verify the session is still active: `GET /api/v1/sessions/{id}`
4. Session state is persisted in MySQL — disconnection does not lose progress.

15.5 OTP Not Received

Symptoms: Guest reports not receiving the verification code email.

Solution:

1. In development, check MailHog at `http://localhost:8025`.
2. Verify the guest's email in the reservation is correct.
3. Trigger a new OTP: the old code may have expired (10-minute window).
4. In production, check your SMTP provider's sending logs.
5. OTP delivery retries 3 times internally before reporting failure.

15.6 Rate Limiting

Symptoms: `429 Too Many Requests` responses.

Solution: Default rate limit is 60 requests/minute per API key. Adjust `RATE_LIMIT` in `.env` if needed.

15.7 Invalid State Transition (422)

Symptoms: `422 Unprocessable Entity` when sending a message.

Solution: The LLM detected an intent that doesn't match the current state (e.g., guest said "I agree" while in `ID_VERIFY_PENDING`). The state machine rejects invalid transitions. The agent should clarify the required action. If the issue persists, check the state via `GET /sessions/{id}/state`.

16. Glossary

Term	Definition
Booking Reference	A unique code (e.g. <code>BK-2024-001</code>) identifying a reservation in the system.

Term	Definition
Session	A check-in session linking a guest to a reservation. Tracks the current onboarding state.
Session Token	A time-limited token (24h) generated per session for authenticating guest-facing endpoints (OTP, ID upload, payment).
State Machine	The deterministic workflow engine with 9 states that enforces the onboarding sequence. The single source of truth for guest progress.
Intent	A classified guest action: <code>agree</code> , <code>decline</code> , <code>confirm</code> , <code>provide_info</code> , <code>upload_id</code> , <code>select_option</code> , <code>question</code> , <code>resume</code> , <code>start</code> .
MCP Tool	An internal Python function exposed to the LLM agent (e.g. <code>record_agreement</code> , <code>trigger_otp</code>). The agent calls tools; the state machine validates results.
OTP	One-Time Password — a 6-digit code emailed to the guest for identity verification. Expires in 10 minutes, max 3 attempts.
Incidental Protection	Damage Waiver (\$49.00, non-refundable) or Security Hold (\$250.00, refundable) selected during onboarding.
Secure Link	A time-limited (1 hour), token-authenticated URL for ID upload or incidental protection selection.
Audit Trail	An immutable log of every state transition, tool call, and guest action with timestamps.
Ollama	Local LLM runtime that serves the Llama 3.1 8B model for intent detection and conversation.
MailHog	A development SMTP server that captures emails for testing (Web UI at port 8025).
Pydantic	Python data validation library used for request/response schemas and settings management.
Alembic	Database migration tool that version-controls schema changes for SQLAlchemy.

17. Appendix: Database Schema

10 tables managed by SQLAlchemy ORM with Alembic migrations.

reservations

Column	Type	Constraints
id	VARCHAR(36)	PK, UUID default
booking_reference	VARCHAR(50)	UNIQUE, NOT NULL, INDEXED
property_name	VARCHAR(255)	NOT NULL
property_address	TEXT	NOT NULL
guest_name	VARCHAR(255)	NOT NULL
guest_email	VARCHAR(255)	NOT NULL
guest_phone	VARCHAR(50)	NULLABLE
check_in_date	DATE	NOT NULL
check_out_date	DATE	NOT NULL
num_guests	INTEGER	DEFAULT 1

Column	Type	Constraints
wifi_network	VARCHAR(255)	NULLABLE
wifi_password	VARCHAR(255)	NULLABLE
lockbox_code	VARCHAR(50)	NULLABLE
emergency_contact	VARCHAR(255)	NULLABLE
house_rules_text	TEXT	NOT NULL
rental_agreement_text	TEXT	NOT NULL
privacy_policy_text	TEXT	NOT NULL
created_at	DATETIME	DEFAULT now()
updated_at	DATETIME	DEFAULT now(), onupdate now()

guests

Column	Type	Constraints
id	VARCHAR(36)	PK, UUID default
email	VARCHAR(255)	UNIQUE, NOT NULL, INDEXED
first_name	VARCHAR(255)	NOT NULL
last_name	VARCHAR(255)	NOT NULL
phone	VARCHAR(50)	NULLABLE
id_document_path	VARCHAR(255)	NULLABLE
id_verified	BOOLEAN	DEFAULT FALSE
created_at	DATETIME	DEFAULT now()
updated_at	DATETIME	DEFAULT now(), onupdate now()

sessions

Column	Type	Constraints
id	VARCHAR(36)	PK, UUID default
reservation_id	VARCHAR(36)	FK → reservations.id, NOT NULL
guest_id	VARCHAR(36)	FK → guests.id, NOT NULL
current_state	VARCHAR(50)	DEFAULT 'INIT'
session_token	VARCHAR(255)	UNIQUE, NOT NULL, INDEXED
status	VARCHAR(20)	DEFAULT 'active'
last_message_at	DATETIME	NULLABLE
created_at	DATETIME	DEFAULT now()
updated_at	DATETIME	DEFAULT now(), onupdate now()

agreements

Column	Type	Constraints
id	VARCHAR(36)	PK, UUID default
session_id	VARCHAR(36)	FK → sessions.id, NOT NULL
agreement_type	VARCHAR(50)	NOT NULL (privacy_policy / house_rules / rental_agreement)
accepted	BOOLEAN	NOT NULL
guest_response	VARCHAR(500)	NULLABLE
created_at	DATETIME	DEFAULT now()

otp_verifications

Column	Type	Constraints
id	VARCHAR(36)	PK, UUID default
session_id	VARCHAR(36)	FK → sessions.id, NOT NULL
email	VARCHAR(255)	NOT NULL
otp_code	VARCHAR(10)	NOT NULL, INDEXED
verified	BOOLEAN	DEFAULT FALSE
attempts	INTEGER	DEFAULT 0
max_attempts	INTEGER	DEFAULT 3
expires_at	DATETIME	NOT NULL
created_at	DATETIME	DEFAULT now()

incidental_selections

Column	Type	Constraints
id	VARCHAR(36)	PK, UUID default
session_id	VARCHAR(36)	FK → sessions.id, NOT NULL
selection_type	VARCHAR(50)	NOT NULL (damage_waiver / security_hold)
amount	FLOAT	NOT NULL
payment_status	VARCHAR(20)	DEFAULT 'pending'
payment_reference	VARCHAR(255)	NULLABLE
created_at	DATETIME	DEFAULT now()

messages

Column	Type	Constraints
id	VARCHAR(36)	PK, UUID default

Column	Type	Constraints
session_id	VARCHAR(36)	FK → sessions.id, NOT NULL
role	VARCHAR(20)	NOT NULL (guest / agent / system)
content	TEXT	NOT NULL
intent_detected	VARCHAR(50)	NULLABLE
tools_called	JSON	NULLABLE
created_at	DATETIME	DEFAULT now()

audit_trail

Column	Type	Constraints
id	VARCHAR(36)	PK, UUID default
session_id	VARCHAR(36)	FK → sessions.id, NOT NULL
action	VARCHAR(100)	NOT NULL
from_state	VARCHAR(50)	NULLABLE
to_state	VARCHAR(50)	NULLABLE
details	JSON	NULLABLE
actor	VARCHAR(20)	NOT NULL (guest / agent / system)
created_at	DATETIME	DEFAULT now()

knowledge_base

Column	Type	Constraints
id	VARCHAR(36)	PK, UUID default
property_id	VARCHAR(50)	NOT NULL, INDEXED
question	TEXT	NOT NULL
answer	TEXT	NOT NULL
category	VARCHAR(50)	NULLABLE
created_at	DATETIME	DEFAULT now()
updated_at	DATETIME	DEFAULT now(), onupdate now()

api_keys

Column	Type	Constraints
id	VARCHAR(36)	PK, UUID default
key	VARCHAR(255)	UNIQUE, NOT NULL, INDEXED
name	VARCHAR(255)	NOT NULL

Column	Type	Constraints
is_active	BOOLEAN	DEFAULT TRUE
created_at	DATETIME	DEFAULT now()